

Lecture 1

1. Definitions

- **Program** = A complete, executable implementation of one or more algorithms and their chosen data structures, written in a programming language.
 - **Algorithm** = Well-defined step-by-step procedure that takes input → produces output.
 - **Data Structure** = A systematic way to organize, manage, and store data so that it can be accessed and modified efficiently.
-

2. Steps to Develop an Algorithm

1. Understand the problem (inputs & outputs).
 2. Break down (decompose) the problem.
 3. Plan using pseudocode or flowcharts.
 4. Identify edge cases.
 5. Refine → remove redundancy & optimize.
 6. Test with normal & edge cases.
-

3. Algorithm Design Techniques

1. **Iterative (Loops):** Repeat steps until done. Example: check every item in a list.
 2. **Recursive:** Solve problem by calling smaller versions of itself. Example: folder search.
 3. **Greedy:** Always take the best current choice. Example: choose shortest line at a store.
 4. **Divide & Conquer:** Split → solve parts → combine. Example: Merge Sort.
 5. **Dynamic Programming:** Divide & Conquer + memory (store solutions to subproblems). Example: shortest path in a map.
-

4. Algorithm Analysis Criteria

- **Correctness:** Always produces the right output.
- **Performance:** Efficient in time & space.
- **Simplicity:** Easy to implement, debug, and maintain.

5. Algorithmic Performance

Time Complexity

- Measures **speed** of an algorithm.
- Affected by: number of inputs, type of operations, loops, recursion.

Space Complexity

- Measures **memory usage**.
- Affected by: chosen data structures, extra storage.

👉 In exams: **Time complexity is the main focus.**

Lecture 2

1. Pseudo-Code

- Informal way to describe algorithms, uses programming-like structure but meant for humans.

2. Low-Level Analysis

- Count **primitive operations** (independent of programming language):
 - Arithmetic operation (+, -, etc.)
 - Array indexing ($A[i]$)
 - Comparisons ($x < y$)
 - Function calls / returns

3. Types of Analysis

- **Worst Case** → maximum running time (most common, gives upper bound).
 - **Best Case** → minimum running time (not very useful).
 - **Average Case** → expected time, harder to compute (needs probability distribution).
-

4. General Rules

- **Loops:** running time = #iterations × cost of body.
 - **Nested loops:** multiply loop sizes.
 - **Consecutive statements:** add them.
 - **If/Else:** take the larger cost branch.
-

5. Growth-Rate Functions

- Expressed as Big-O.
-

6. Summation Rules

Rule 1: Sum of Constant 1

$$\sum_{i=1}^n 1 = n$$
$$T(n) = O(n)$$

Rule 2: Sum of Constant C

$$\sum_{i=1}^n C = C \cdot n$$
$$T(n) = O(n)$$

Rule 3: Sum of Index Variable (Arithmetic Series)

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$
$$T(n) = O(n^2)$$

Rule 4: General Polynomial Sum Rule

$$\sum_{i=1}^n i^k = O(n^{k+1}) \quad \text{for } k \geq 0$$

Rule 5: Logarithmic Number of Terms

$$\sum_{i=1}^{\log_2 n} C = C \cdot \log_2 n$$
$$T(n) = O(\log n)$$

Rule 6: Harmonic Series

$$\sum_{i=1}^n \frac{1}{i} = O(\log n)$$

Lecture 3

1. Recursion vs Loops

- Both recursion and iteration can solve same problems.
- **Recursion = easier for programmer** (مريح للمبرمج)
- **Loops = faster for computer** (أسرع للأداء)

→ Each recursive call is stored in the **Call Stack** (المكدس).

Base Case & Recursive Case

Every recursive function must have:

- **Base Case:** when recursion stops.
- **Recursive Case:** when function calls itself.

Example:

```
def countdown(i):  
    print(i)  
    if i <= 0: # Base case  
        return  
    else:      # Recursive case  
        countdown(i - 1)
```

3. Recursive Function Form

General recurrence:

$$T(n) = aT(\text{smaller size}) + b$$

Where:

- **a** = number of recursive calls
 - **b** = cost of non-recursive part
-

4. Examples of Recurrence Relations

Example	Recurrence	Time Complexity
Basic recursion	$T(n) = T(n-1) + 2n + 2$	$O(n^2)$ (grows with n^2)
Factorial simple	$T(n) = T(n-1) + 1$	$O(n)$
Factorial modified	$T(n) = 4T(n-1) + 1$	$O(4^n)$ (exponential)
Divide by 2	$T(n) = T(n/2) + 1$	$O(\log n)$
Double recursion	$T(n) = 2T(n/2) + 1$	$O(n)$
Fibonacci	$T(n) = T(n-1) + T(n-2) + 1$	$O(2^n)$

5. Tower of Hanoi Problem

Rules:

1. Move all disks from **A** → **C**, using **B** as spare.
2. Only one disk at a time.
3. Larger disk can't be on smaller one.

Recurrence:

$$T(n) = 2T(n-1) + 1$$

Base case: $T(0) = 1$

→ Pattern:

n Moves

1 1

2 3

3 7

4 15

Formula:

$$T(n) = 2^n - 1 \Rightarrow O(2^n)$$

6. Geometric Series (المتسلسلة الهندسية)

General Form:

$$S = a + ar + ar^2 + ar^3 + \dots + ar^{n-1}$$

Sum Formula:

$$S_n = a \frac{r^n - 1}{r - 1}$$

If $a = 1$, $r = 2$ (Tower of Hanoi):

$$S = 2^n - 1$$

7. Substitution Method (طريقة التعويض)

Example 1 – Tower of Hanoi

$$T(n) = 2T(n-1) + 1$$

Keep substituting:

$$\rightarrow T(n) = 2^n - 1$$

 $O(2^n)$

Example 2 – Binary Search

$$T(n) = T(n/2) + 1$$

Substitute until $n/2^k = 1 \rightarrow k = \log_2 n$

$$\rightarrow T(n) = 1 + \log_2 n$$

 $O(\log n)$

8. Master Theorem (نظرية السيد)

Form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Compare $f(n)$ with $n^{\log_b a}$

3 Cases of the Master Theorem

For recurrences of the form:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + n^d \log^k n$$

We can simplify the comparison:

1. If $n^{\log_b a} > n^d$, then $T(n) = O(n^{\log_b a})$
2. If $n^{\log_b a} = n^d$, then $T(n) = O(n^d \log^{k+1} n)$
3. If $n^{\log_b a} < n^d$, then $T(n) = O(n^d \log^k n)$

Examples

Example	Recurrence	Result
1	$T(n) = 4T(n/2) + n$	$O(n^2)$
2	$T(n) = 4T(n/2) + n^2$	$O(n^2 \log n)$
3	$T(n) = 4T(n/2) + n^3$	$O(n^3)$
4	$T(n) = 9T(n/3) + n$	$O(n^2)$
5	$T(n) = 2T(n/2) + n$	$O(n \log n)$ (Merge Sort)
6	$T(n) = T(n/2) + 1$	$O(\log n)$
7	$T(n) = 2T(n/4) + n^2 \log^2 n$	$O(n^2 \log^2 n)$

Final Quick Notes for Exam

- Always find **a, b, f(n)** in the recurrence.
- Compare **f(n)** with **$n^{\log_b a}$** .
- For recursion like $T(n) = T(n/2) + 1 \rightarrow \log n$.
- For $T(n) = 2T(n/2) + n \rightarrow n \log n$.
- For $T(n) = 2T(n-1) + 1 \rightarrow 2^n$.
- For Fibonacci $\rightarrow 2^n$.